ENGENHARIA · DESENVOLVIMENTO ASSISTIDO POR AGENTES

Construindo um app multiplataforma com Claude Code

Regras de projeto, agentes especializados, decisões de arquitetura, testes, release automatizado, MCPs, comunicação entre agentes e uma skill de relatórios — o que funcionou, o que quebrou e o que ficou registrado.

Estudo de caso de um app Expo (iOS, Android e web) com Supabase, integrado a um sistema de gestão externo.
Outubro de 2026. Documento público: sem dados de cliente, sem regras de negócio, sem credenciais.

Sumário

1. Por que este artigo existe
2. O projeto em uma página
3. O CLAUDE.md como contrato do projeto
4. Invariantes: o que é caro demais para errar
5. Fronteiras e direção de dependência
6. Regras de código e como elas são impostas
7. Registrando um padrão novo
8. Como as decisões de arquitetura são tomadas
9. Testes: onde errar é caro
10. CI, versionamento e release
11. Claude Code no projeto: agentes, hooks, memória e disciplina de contexto
12. Servidores MCP
13. papo: quando dois Claudes precisam combinar um contrato
14. A skill de relatórios de proteção de dados e engenharia
15. Lições aprendidas
16. O que levar deste caso

1. Por que este artigo existe

Este documento descreve como um app móvel e web foi construído e mantido por um time pequeno com o Claude Code como par de programação permanente. Não é um tutorial de Expo nem um manual do Claude Code: é o registro das **regras** que o projeto adotou, do **porquê** de cada uma e de como elas são **impostas** por lint, tipo, teste ou CI em vez de depender de memória.

A tese central é simples: um agente de código é tão bom quanto o contrato que ele recebe. Quando o repositório diz, em um lugar só, o que é invariante, onde cada coisa mora e como se verifica que está certo, o agente produz código que parece ter sido escrito por quem já conhece o projeto. Quando isso não existe, cada sessão reinventa uma convenção.

Tudo aqui foi anonimizado. O domínio é descrito de forma genérica, e nenhum identificador de ambiente, credencial ou regra de negócio aparece no texto.

2. O projeto em uma página

O produto é um aplicativo para profissionais autônomos de um setor regulado. Ele é a ponta móvel de um sistema de gestão já existente (chamado aqui de **ERP**), que continua sendo a fonte de verdade do domínio. O app autentica usuários, mostra o que o ERP envia, recebe respostas, guarda documentos de cadastro e oferece um chat com IA que lê documentos do processo e responde com citação de página.

Camada	Escolha
Cliente	Expo SDK 54, React Native, Expo Router (rotas tipadas), React Compiler ligado, TypeScript estrito. Um código para iOS, Android e web.
Backend	Supabase: Postgres com RLS, Auth, Storage, Edge Functions em Deno e Vault para segredos de gatilhos.
Integração	ERP externo por API autenticada e webhooks de eventos, com contrato documentado no repositório.
Observabilidade	Análise de produto e captura de erros por uma porta única; crash reporting nativo.
Suporte	Chat de suporte embutido por SDK nativo, atrás de um shim.
Entrega	Web em hospedagem estática; apps por EAS Build com envio automático às lojas; CI no GitHub Actions.
Ambientes	Dois projetos de backend (homologação e produção) e perfis de build correspondentes.
Time	Um desenvolvedor responsável pelo app, Claude Code como par permanente, e um time separado cuidando do ERP.

3. O CLAUDE.md como contrato do projeto

O `CLAUDE.md` na raiz do repositório é lido pelo agente em toda sessão. No início era uma lista de comandos; virou o documento mais importante do projeto. Ele tem nove partes, sempre nesta ordem:

1. **Mapa de documentos.** Uma linha por documento especializado (regras de plataforma, contrato do ERP, privacidade, backlog, plano de testes, release). O CLAUDE.md aponta; não repete.
2. **Comandos** e, logo abaixo, a **automação que já existe**, para o agente não duplicar: formatação roda sozinha por hook; tipos do banco são gerados; o portão de verificação roda uma vez, no fim.
3. **Invariantes**: o que é caro demais para errar (seção 4).
4. **Como registrar um padrão novo** (seção 7).
5. **Fronteiras**: direção de dependência entre pastas (seção 5).
6. **Convenções**: idioma, comentários, tipagem, tradução de erro, shims, monitoramento, textos legais.
7. **Testes**: quando escrever e quando não escrever.
8. **Armadilhas que o projeto já aprendeu**: cada uma custou pelo menos uma sessão.
9. **Desenho** e **disciplina de contexto**: como decidir onde uma capacidade mora e como gastar tokens.

> **PRINCÍPIO**
> Cada regra no CLAUDE.md responde a três perguntas: o que fazer, qual problema evita e onde é imposta. Uma regra sem o "onde é imposta" é um pedido; com ele, é um contrato. Quando a imposição é só "convenção, sem imposição", o texto diz isso explicitamente.

Dois cuidados mantêm o arquivo útil: ele não repete o que o código ou o git já dizem (estrutura de pastas, histórico de correções), e ele é atualizado no *mesmo commit* em que um padrão muda. Registro velho é pior que registro nenhum, porque o agente confia nele.

4. Invariantes: o que é caro demais para errar

Há bugs que se consertam no próximo PR e bugs cujo dano já aconteceu quando se descobre: vazamento de dado de outro usuário, demanda perdida, prazo calculado errado. Para esses, "a gente arruma depois" não vale. O projeto lista nove invariantes:

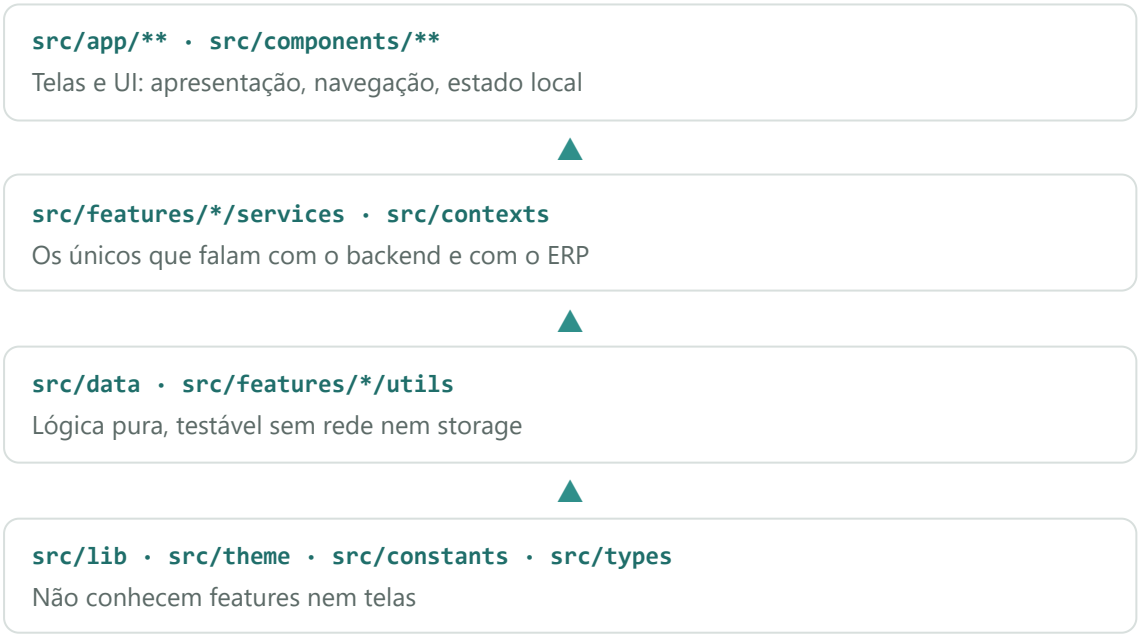
1. **Toda tabela com dado de outro usuário tem RLS por padrão.** Autorização não é "lembrar de filtrar por `user_id` na query". Uma tabela nova sem policy é bug de segurança mesmo sem tela que a leia.
2. **Credencial privilegiada vive só em Edge Function.** Qualquer variável `EXPO_PUBLIC_*` entra no bundle e é pública, inclusive na web, onde a saída é servida estática. Se um segredo precisou virar pública, o desenho está errado. Gatilhos e crons do banco que chamam funções leem URL e segredo do Vault, nunca de literal no DDL, porque a definição do gatilho aparece em todo dump e no painel.

3. **Autorização é no Postgres, não na tela.** Esconder um botão é cortesia; o portão é a policy. Toda regra com caso não trivial (dono, estado, papel) precisa de teste do ator *não* autorizado.
4. **A entidade central do domínio tem ciclo de vida explícito.** Transições inválidas são rejeitadas onde o estado mora, não só onde a tela chama. Há uma rota única de resposta para o sistema externo.
5. **Valor monetário é dado do sistema externo, não conta do app.** O cliente formata; nunca recalcula, arredonda ou infere.
6. **Timestamp vem do servidor, nunca do relógio do aparelho.** Prazo medido com hora local é prazo errado.
7. **Nenhuma cor nem `fontSize` literal** nas pastas de telas, componentes e features. Falta um token? Adiciona-se o token antes, não uma exceção depois.
8. **Migração aplicada não se edita.** Corrige-se com uma nova.
9. **Action de CI presa a tag mutável é buraco de supply chain.** Pinar por SHA e dar ao workflow o mínimo de `permissions:`.

Repare que seis das nove têm imposição mecânica (RLS no banco, lint para tokens, teste para texto legal, CI para audit) e três são convenção reforçada por revisão de agente especializado (seção 11).

5. Fronteiras e direção de dependência

A estrutura de pastas segue uma direção de dependência do mais estável para o mais volátil. Uma camada só importa das que estão abaixo dela.



Direção de dependência: cada camada só importa das que estão abaixo.

- Uma feature não importa de outra feature. O que duas precisam sobe para `src/lib`, `src/components/ui` ou `src/data`.

- `features/*/utils` é lógica pura: sem cliente de banco, sem `fetch`, sem React. É o recorte que está sob cobertura de teste.
- Tela não chama o cliente do banco direto; só `services` e `contexts` fazem isso.
- Um `eslint-disable` numa regra de fronteira significa que a fronteira está errada, não que a exceção está ok.

A regra de "feature não importa de feature" está no ESLint como `no-restricted-imports` com um padrão por feature, e a mensagem do lint aponta para a seção do CLAUDE.md. Hoje ela é *warning*, não erro, porque foi adotada com código legado que ainda a viola; a lista de avisos é a dívida visível, e ela só diminui.

6. Regras de código e como elas são impostas

Regra	Problema que evita	Onde é imposta
Produto em português; identificadores em inglês ou no termo de domínio já usado	Copy misturada e uma terceira convenção de nomes por arquivo	Convenção; revisão por agente
Nenhum comentário no código	Comentário que envelhece e mente; nome ruim escondido atrás de explicação	Convenção; o porquê vai na mensagem de commit ou no PR
Tipagem estrita, sem `any`, `!` ou cast sem checagem	Erro de runtime que o compilador já pegaria	`tsc --noEmit` no CI; justificativa no commit quando inevitável
Um único lugar traduz erro de infra (código do Postgres, status HTTP)	Erro cru chegando na UI por quinze `catch` diferentes	Módulo único por tipo de erro; teste por código
Design tokens: sem hex/rgb/hsl nem `fontSize` numérico; paleta crua só dentro de `src/theme`	Tema escuro quebrado, contraste fora da norma, estilos impossíveis de mudar em lote	`no-restricted-syntax` e `no-restricted-imports` no ESLint; teste de contraste da paleta
SDK nativo de terceiro entra por shim em `src/lib` (`.native.ts` / `.web.ts`), inicializado por um componente headless atrás de `Platform.OS !== 'web'`	Módulo nativo no bundle web e no Jest	`no-restricted-imports` barra o pacote fora de `src/lib`
Erro capturado passa por uma porta única de monitoramento	Dois destinos divergentes; SDK chamado direto em todo `catch`	ESLint barra o SDK e o shim fora de `src/lib`

Texto legal tem versão presa ao conteúdo (SHA-256); o banco só aceita versão cadastrada com o mesmo hash e grava cada aceite em tabela só de inserção	Usuário aparecendo como tendo aceitado um texto que não viu	Teste que compara texto × hash × migração; RPC que rejeita hash divergente
Edge Function chamada pelo app autentica com o helper compartilhado, nunca com cópia local	Verificação de JWT divergente entre funções	Convenção, sem imposição

A exceção registrada merece menção: o SDK de crash reporting inicializa no escopo do módulo do layout raiz, não num componente headless, porque um `useEffect` só roda depois do primeiro render e perderia crash de boot. A exceção está escrita ao lado da regra, com o motivo.

7. Registrando um padrão novo

Adotou um padrão? Ele é escrito no mesmo commit que o introduz. Sem registro, o padrão sobrevive até a próxima pessoa (ou o próximo agente) abrir o arquivo e escolher outra coisa; aí são dois jeitos de fazer a mesma coisa.

Um registro tem quatro partes: **o que fazer**, **o problema que evita**, **onde é imposto** (lint, tipo, teste, CI, ou "convenção, sem imposição") e **um arquivo de exemplo**. Três consequências práticas:

- Automatizar a imposição quando der. Neste repositório, regras de design token e de import viram regras no `eslint.config.js`, não lembrete em revisão.
- Aplicar o padrão ao código existente no mesmo commit. Padrão que só vale para arquivo novo cria duas convenções em vez de uma.
- Quando o padrão mudar, atualizar o registro.

> **EXEMPLO RECENTE**
>
> O Android com *edge-to-edge* desenha o app por baixo da barra de navegação do sistema. Conteúdo preso à base com padding fixo ficava escondido. A correção virou um hook (`useBottomSafePadding(base)`) que soma a área segura só no Android, foi aplicada a todos os pontos encontrados por uma varredura de agente, ganhou mock no setup do Jest e um registro em `docs/mobile-rules.md` com as quatro partes: o que, por quê, onde é imposto (convenção) e exemplo.

8. Como as decisões de arquitetura são tomadas

O projeto não tem um diretório de ADRs. Tem algo mais barato e, na prática, mais usado: uma **escada de onde uma capacidade mora**, três critérios de corte e o hábito de escrever o porquê na mensagem de commit. Esta seção elabora cada um e mostra decisões reais tomadas com eles.

8.1 A escada: desça um nível só quando o de cima genuinamente não serve

1. **Banco**, se é invariante que vale para todo cliente da API: unicidade, transição de estado, autorização, agregado materializado.
2. **Lógica pura** (`features/*/utils` , `src/data`), se é regra de negócio testável sem rede nem storage.
3. **Camada de acesso** (`services` , cliente da API externa), se é query, mapeamento ou tradução de erro.
4. **Tela**, se é apresentação, navegação ou estado local.

A escada força a pergunta certa antes de escrever código: "se outro cliente (um script, um painel, uma função agendada) tocar nesse dado, a regra ainda vale?" Se a resposta é sim, a regra é do banco. Isso explica por que autorização, ciclo de vida e versão de texto legal vivem em policies, triggers e RPCs, e por que a tela só reflete o que o banco já garante.

8.2 Três critérios de corte

- **YAGNI ganha por padrão.** Duas ocorrências não justificam abstração; três em contextos genuinamente idênticos, sim. Contexto diferente não é duplicação: não se unifica código que só parece igual hoje e vai divergir amanhã por motivos não relacionados.
- **Estado inválido irrepresentável** vence validação espalhada. Uma união discriminada de estados de resposta (`respondida | insuficiente | sem_documento | nao_indexado`) faz o compilador cobrar cada caso em vez de depender de um `if` em cada tela.
- **Se entender um desenho novo exige abrir mais de dois arquivos, o desenho perdeu**, mesmo estando tecnicamente correto.

8.3 Decisões concretas e seus trade-offs

AUTORIZAÇÃO NO POSTGRES COM RLS

Alternativa descartada: filtrar por usuário nas queries do app. Motivo: todo cliente novo (painel, função, script de migração de dados) repetiria o filtro ou esqueceria. Custo aceito: toda tabela nova precisa de policy e de teste do ator não autorizado, e erros de permissão são mais difíceis de diagnosticar ("retorna zero linhas" em vez de 403). Um agente dedicado ao banco existe em parte por isso.

SEGREDOS SÓ EM EDGE FUNCTIONS, E VAULT PARA GATILHOS

Uma versão anterior tinha a chave privilegiada do banco dentro da definição de um gatilho que chamava uma função de push. Funcionava, mas a definição do gatilho aparece em todo dump e no painel. A decisão foi mover URL e segredo para o Vault e fazer o gatilho ler de lá. O custo é um passo a mais no provisionamento de cada ambiente; o ganho é que um dump do esquema deixa de ser um vazamento.

RELÓGIO DO SERVIDOR POR RPC, EM PLPGSQL

O app precisa de "agora" para contagens regressivas, e o relógio do aparelho não é confiável. A solução é uma função `server_now()` no banco. Detalhe que custou uma investigação: a primeira versão era SQL puro e o planejador a *inlinava*; um plano preparado por um papel autenticado e reaproveitado por um anônimo pulava a checagem de `EXECUTE`. A função foi reescrita em plpgsql, que não é inlinado, e o motivo está no comentário da função no banco (o único lugar em que o projeto escreve comentário, porque o DDL não tem mensagem de commit).

CICLO DE VIDA DA ENTIDADE CENTRAL COM ROTA ÚNICA

O sistema externo recebia aceite e recusa por duas rotas. O contrato foi consolidado numa rota única de resposta, com anotação estruturada, e as transições passaram a ser rejeitadas onde o estado mora. Trade-off assumido: o app ainda precisa aceitar, por um tempo, uma transição que o operador humano do ERP pode registrar fora do prazo; isso está no backlog como decisão pendente do outro time, não como bug.

TEXTO LEGAL COM VERSÃO, HASH E TABELA SÓ DE INSERÇÃO

Quando um termo muda, o que o usuário aceitou precisa ser provável depois. A escolha foi: o texto vive no código com versão e SHA-256; uma migração cadastra o par (versão, hash); uma RPC rejeita aceite cujo hash não bate; a tabela de aceites só aceita inserção. Um teste compara o texto exibido, o hash declarado e o SQL da migração. O custo é um ritual a cada mudança de texto (nova versão, nova migração aplicada *antes* do deploy do app, todos aceitam de novo). O ganho é não precisar explicar para um advogado por que o registro de aceite não bate com o texto.

SHIM PARA SDKS NATIVOS

Três SDKs nativos (suporte, mensageria, crash) entram por um arquivo em `src/lib` com variantes `.native.ts` e `.web.ts`. O Metro escolhe pelo sufixo; o Jest e a web nunca carregam o módulo nativo. A inicialização é um componente headless no layout raiz. O custo é manter a superfície do shim em dia com a do SDK; o ganho é um bundle web limpo e testes que não precisam mockar módulo nativo um por um.

NAVEGAÇÃO NATIVA COM DOIS LAYOUTS

A barra de abas usa o componente nativo (alpha no SDK atual) no iOS e no Android, e o componente em JavaScript na web. São dois arquivos de layout escolhidos por sufixo, e cada aba visível é uma pilha própria com cabeçalho nativo. Trade-off: rota nova nas abas entra nos dois arquivos, e o Metro não avisa se um ficar para trás. A armadilha está registrada; o ganho (gestos, grandes títulos e efeito de vidro nativos) foi considerado maior.

CACHE PERSISTIDO COM LIMPEZA NO LOGOUT

As consultas são persistidas no aparelho para abrir rápido sem rede. A decisão de desenho que importa é a de *apagar*: ao sair da conta, o cache é limpo antes de qualquer tela montar, para um segundo usuário do mesmo aparelho nunca ver dado do primeiro. Um hook de "ainda restaurando" evita render com cache de outra sessão.

GATE DE AUDITORIA DE DEPENDÊNCIAS COM ALLOWLIST

`npm audit` cru bloqueia o CI por vulnerabilidades sem correção disponível. A solução é um script que falha só em *high/critical* fora de uma allowlist documentada, e ignora cópias que vivem apenas dentro das dependências empacotadas do próprio CLI do npm. Adicionar item à allowlist é decisão, não conserto: exige justificativa escrita. No dia em que duas vulnerabilidades críticas foram publicadas numa dependência transitiva do gerador de release, o gate parou a publicação até a atualização de quatro linhas no lockfile.

O PORQUÊ VAI NO COMMIT, NÃO NO CÓDIGO

Com a regra de "nenhum comentário", o histórico do git virou o registro de decisões. Cada mensagem de commit tem um título em Conventional Commits e um corpo que explica a alternativa descartada quando houve uma. É um ADR leve: pesquisável por `git log --grep`, nunca desatualizado em relação ao código que acompanha.

> #### DECISÕES QUE O PIPELINE DE RELEASE ENSINOU
>
> **Back-merge após cada release.** A versão de marketing só sobe na branch principal. Um build de homologação gerado a partir da branch de desenvolvimento saía com a versão anterior, e a loja o recusou porque aquela versão já estava fechada. Hoje a branch principal volta para a de desenvolvimento depois de cada release, sem commit novo (fast-forward).
>
> **Nome exibido, não nome do projeto.** Para distinguir o build de homologação no aparelho, a primeira tentativa trocou o `name` do app por perfil de build. Isso renomeou o alvo do Xcode, e o serviço de build falhou ao atribuir o perfil de provisionamento. A solução correta muda só o nome exibido (`CFBundleDisplayName` no iOS, `app_name` no Android), por config plugin, lendo o perfil de build do ambiente.
>
> **Jobs que herdam o skip.** No GitHub Actions, um job sem `if` próprio é pulado quando qualquer job anterior na cadeia foi pulado. Os jobs de build mobile dependiam de um job de release que é sempre pulado fora da branch principal, então o disparo manual de homologação nunca gerava build. Cada job de build ganhou um `if` que olha só para o job que resolve o perfil.

9. Testes: onde errar é caro

Teste existe onde errar é caro; porcentagem de cobertura não é objetivo. O `collectCoverageFrom` do projeto mostra o recorte real: utilitários puros das features centrais, o cliente da API externa, o módulo de status e o tema. O CLAUDE.md diz quando escrever e quando não:

ESCREVA TESTE QUANDO

- uma regra de autorização tem caso não trivial, e o teste é do ator *não* autorizado;
- uma transição de estado é adicionada: caso válido e caso rejeitado;
- um prazo ou contagem é calculado;
- uma regra está duplicada de propósito entre app e banco: o teste compara os dois mecanicamente.

NÃO ESCREVA TESTE PARA

- getter;
- mapeamento direto linha → DTO;
- o que o sistema de tipos já garante.

9.1 Os níveis que existem de verdade

Nível	Ferramenta	O que cobre
Unitário	Jest com preset do Expo	Lógica pura (prazos, contagens, normalização de respostas, classificação de perguntas), paleta (contraste mínimo por par de tokens), componentes com mocks de módulos nativos, textos legais (texto × hash × migração).
Ponta a ponta web	Playwright contra o ambiente de homologação real, com o sistema externo mockado por rota	Fluxos de login, cadastro, listas, uploads, notificações; e uma suíte de RLS que cria dois usuários e prova que um não lê o outro.
Carga	k6, com limiares declarados	Leitura concorrente, rota de resposta, lote de webhooks; alvos registrados como regra (p95 por rota, tamanho de lote).
Sondas de segurança e concorrência	Scripts Node contra as Edge Functions e o banco de homologação	Método e segredo errados, payload inválido, limite de lote, replay de evento, aceite duplicado, chamadas paralelas, relógio.
No aparelho	Roteiro humano, com número de build	Push, deep link, resiliência sem rede, acessibilidade.

9.2 Um plano de testes executável por agente

O projeto tem um plano de quinze dias escrito para ser *executado pelo agente*, não lido por uma pessoa. Ele abre com instruções operacionais: o ambiente alvo (sempre homologação; produção só leitura, e só onde o teste disser), como provar o alvo antes da primeira escrita do dia, como obter credenciais sem jamais imprimi-las, o prefixo dos dados de teste e a limpeza obrigatória, o formato do

registro de resultados (uma linha por execução: ID, data, resultado, evidência, observação), quando parar e pedir decisão humana (falha de segurança ou de concorrência interrompe a sessão) e a escala de severidade.

Três agentes dedicados sustentam o plano: um que decide a estratégia e o dimensionamento, um que revisa executabilidade, determinismo e isolamento do código de teste, e um que transforma o modelo de ameaças em sondas. Os resultados vivem num documento versionado, e cada bug encontrado vira item de backlog com o ID do teste.

10. CI, versionamento e release

10.1 O portão único

`npm run build` é a verificação completa: lint, `expo-doctor`, export da web, typecheck, checagem das Edge Functions em Deno, testes e o gate de auditoria. Roda uma vez, no fim de uma tarefa; não depois de cada edição. O CI roda o mesmo comando, então "passou aqui" e "passou lá" significam a mesma coisa.

10.2 Versão a partir dos commits

A versão de marketing vem do `semantic-release`, que lê os commits na branch principal desde a última tag e aplica Conventional Commits. O número de build é da plataforma de build, incrementado automaticamente, e ninguém precisa alinhar os dois.

Prefixo	Efeito
`fix:`	patch
`feat:`	minor
`feat!:` ou rodapé `BREAKING CHANGE:`	major
`chore:`, `docs:`, `refactor:`, `test:`, `ci:`	nenhum: não gera release nem build de loja

Um commit vazio com `fix:` é a forma oficial de "gerar uma versão nova sem mudar nada", por exemplo para abrir um teste externo na loja com um número distinto do que está em análise.

10.3 O que acontece no push na branch principal

1. Job de qualidade: o portão único.
2. Job de release: calcula a versão; se houver bump, atualiza os manifestos e o changelog, commita com `[skip ci]`, cria a tag e a Release.
3. Deploy da web a partir do commit da tag.
4. Builds mobile com perfil de produção e envio automático às lojas, só se houve release.

Push na branch de desenvolvimento roda o portão e publica a web de homologação. Build mobile de homologação é manual, por `workflow_dispatch` com o perfil correspondente, e o app resultante aparece no aparelho com outro nome.

Duas regras de processo completam o fluxo: a PR de desenvolvimento para principal é fundida com *merge commit* (com squash, só o título da PR decidiria o bump), e um job comenta na PR a **prévia da versão** que o merge vai gerar, calculada com `semantic-release` em modo dry-run. Esse job precisou de um ajuste: dentro de uma PR o GitHub informa o branch como `refs/pull/N/merge`, e o `semantic-release` concluía que não estava na branch de publicação. O script passou a ignorar as variáveis de ambiente do CI que descrevem o ref e a usar o branch do checkout.

10.4 Supply chain

Actions pinadas por SHA, `permissions:` mínimo por job, `npm ci` (nunca `npm install`) e o gate de auditoria com allowlist documentada. O hook de pré-commit roda lint e formatação só no que está em stage.

11. Claude Code no projeto: agentes, hooks, memória e disciplina de contexto

11.1 Agentes especializados

O repositório define agentes em `.claude/agents/*.md`. Cada um tem uma descrição que diz *quando* usá-lo e, tão importante quanto, *o que ele não faz* e quem faz. Essa negativa explícita é o que impede o agente principal de pedir revisão de SQL a quem revisa código de aplicação, ou de pedir desenho a quem só revisa diff.

Agente	Papel	Ferramentas
`architect`	Decide desenho antes de o código existir: onde uma capacidade mora, se vale abstrair, se uma fronteira precisa mover.	Leitura e busca
`code-reviewer`	Revisa código já escrito, antes do commit: duplicação real, carga cognitiva, nomes, aderência às convenções do projeto.	Leitura e busca
`db-guardian`	Escreve e revisa tudo da camada de banco: migrações, RLS, policies, gatilhos, funções e seus testes. Diagnostica "retorna zero linhas".	Leitura, escrita, shell
`security-reviewer`	Revisa mudanças que tocam autenticação, privilégio, segredos, dependências ou workflow de CI.	Leitura e busca
`test-writer`	Decide o que merece teste e escreve os que faltam; em bug, o teste que falha primeiro.	Leitura, escrita, shell

`ux-auditor` / `ux-fixer`	Um audita fluxos e copy contra heurísticas e escreve um plano em lotes; o outro executa um lote exatamente como escrito, sem decidir nada. Vários fixers rodam em paralelo.	Auditor: leitura e escrita de doc. Fixer: edição.
`device-qa`	Roda o app num aparelho conectado, tira screenshot, toca, lê o que diverge. Não escreve código.	Shell (adb)
`architect-testes`, `code-reviewer-testes`, `security-reviewer-testes`	Estratégia, revisão e sondas de segurança do plano de testes executável.	Leitura e busca
`context-guard`	Audita contexto e encolhe saídas de ferramenta.	Todas

Três escolhas de desenho dos agentes: ferramentas mínimas por papel (revisores não editam), modelo por papel (o executor de lotes de UX usa um modelo mais barato, porque não decide), e origem comum: as definições vêm de um repositório interno de padrões e são **copiadas e adaptadas**, nunca linkadas, para que cada projeto possa divergir sem quebrar o outro.

11.2 Hooks e permissões

- **PostToolUse**: todo arquivo editado ou escrito passa pelo formatador do projeto. Motivo: sem isso, cada mudança vira um ciclo "editar → rodar lint → achar aspas erradas → editar de novo", e formatação nunca deve custar um turno de raciocínio. O hook é silencioso por contrato: uma falha nele nunca interrompe o agente.
- **permissions.deny**: o agente não lê lockfiles, `node_modules`, saídas de build, cobertura nem arquivos `.env`. Um `.claudeignore` reforça a lista para busca.
- **Modo automático com classificador**: ações que o classificador considera "aplicar às cegas" (por exemplo, aplicar migração num banco) param e pedem decisão. O agente é instruído a não contornar; ele termina o resto da tarefa e explica o que ficou pendente.

11.3 Disciplina de contexto

A seção mais pragmática do CLAUDE.md: ler o trecho, não o arquivo; nunca ler gerado nem vendorado; não reler o que acabou de editar; busca ampla vai para um subagente que devolve uma conclusão, não uma pilha de arquivos; não colar diff grande na resposta; rodar a verificação completa uma vez; saída grande de ferramenta entra resumida (contagem e tipos de falha, não as quinhentas linhas).

Na prática, uma sessão típica delega a varredura ("onde mais isso pode acontecer?") a um agente de exploração em modo somente leitura, recebe uma tabela com arquivo, linha e gravidade, e aplica as correções em lote com um script, em vez de abrir quatorze arquivos um por um.

11.4 Memória persistente

Além do CLAUDE.md (que é do projeto e versionado), o agente mantém uma memória local por projeto, com um arquivo por fato e um índice de uma linha por memória. Os tipos são: *feedback* (como o usuário quer que o trabalho seja feito, com o porquê), *project* (contexto que não dá para deduzir do

código, com datas absolutas), *reference* (ponteiros externos) e *user*. Exemplos de feedback que viraram regra: nenhum comentário no código; nenhuma coautoria do agente em commit, PR ou comentário; ser breve nas respostas; nunca imprimir segredos. Exemplos de projeto: quais são os dois ambientes e qual o CLI está apontando; que o merge na branch principal manda o Android direto à trilha de produção.

11.5 Skills e workflows multiagente

Uma *skill* é um procedimento empacotado (um `SKILL.md` com instruções, mais referências, scripts e modelos) que o agente carrega quando a tarefa bate com a descrição. O projeto usou a skill de criação de skills para transformar um processo longo em uma skill reutilizável (seção 14). Para tarefas que exigem dezenas de agentes com dependências entre eles, o projeto usa *workflows*: scripts que orquestram `agent()`, `parallel()` e `pipeline()` de forma determinística, com fases nomeadas e saída tipada por esquema.

12. Servidores MCP

O Model Context Protocol permite acoplar ferramentas externas ao agente. O projeto declara os servidores em um `.mcp.json` versionado, para que qualquer pessoa que clone o repositório tenha o mesmo conjunto.

Servidor	Para quê
git	Operações de git como ferramentas tipadas (status, diff, log, blame), com integração à interface de permissões.
memory	Grafo de conhecimento com entidades e relações, complementar à memória por arquivos.
playwright	Navegador controlado pelo agente: abrir a web do app, tirar snapshot de acessibilidade, clicar, ler console e rede. Usado para verificar deploys e reproduzir bugs de layout.
codegraphcontext	Grafo de código indexado: achar chamadores, código morto, funções mais complexas, simular impacto de uma mudança.
tree-sitter	Análise sintática por linguagem, para consultas estruturais.
sequential-thinking	Raciocínio em etapas revisáveis para problemas com muitas dependências.
papo	Comunicação direta entre agentes de máquinas diferentes (seção 13).

LIÇÕES DE SETUP NO WINDOWS

O servidor de grafo de código não precisa de um banco de grafos externo; o backend embutido basta. O servidor de tree-sitter exige uma versão específica do SDK de MCP e quebra com a mais nova. O servidor de Playwright responde ao handshake antes de o navegador estar pronto, então a primeira chamada pode falhar sem que o servidor esteja "fora". Cada uma dessas três custou mais de uma hora e está registrada na memória do agente.

13. papo: quando dois Claudes precisam combinar um contrato

O app conversa com um sistema externo mantido por outro time, em outro repositório, em outra máquina. Toda vez que um contrato muda (formato de um evento, motivo de erro novo, status HTTP de uma falha), alguém vira proxy: copia a pergunta de um Claude, cola no chat, o colega cola no Claude dele, copia a resposta e manda de volta.

O **papo** (github.com/Kelvin-Jesus/papo, MIT) resolve isso. É um binário único, sem servidor para hospedar, que conecta duas instâncias do Claude Code ponto a ponto, com criptografia de ponta a ponta e fila para mensagens offline. Para o agente, ele aparece como um servidor MCP; com os *channels* do Claude Code, a mensagem do colega chega sozinha na sessão e o agente reage sem que alguém digite.

13.1 Como foi configurado

1. O próprio agente instalou o binário seguindo a seção "Instruções para o agente" do README do projeto, conferindo o hash do arquivo baixado.
2. Criou uma identidade com nome e entrou numa sala por convite recebido do colega. Quando a sala foi trocada, a identidade antiga foi substituída.
3. Registrou o servidor MCP no escopo do projeto (`claude mcp add --scope local papo --<caminho> mcp`).
4. A sessão com push de mensagens é aberta pelo usuário, com a flag de canais de desenvolvimento apontando para o servidor.

13.2 O que ele muda na prática

Uma negociação típica: o agente do app avisa que uma falha definitiva do webhook passará a voltar como 207 em vez de 500 e que um id inválido vira 400; o agente do outro lado lê o próprio handler, responde o que precisa mudar, e os dois fecham o contrato. Os humanos recebem um resumo e só são chamados quando há decisão a tomar. O servidor instrui o agente a escrever mensagens que se explicam sozinhas, a não vazar segredo para o outro lado e a não entrar em loop de "ok, obrigado".

14. A skill de relatórios de proteção de dados e engenharia

Em um momento do projeto foi preciso entregar dois documentos formais: uma análise de proteção de dados pessoais, para um advogado encaminhar a uma consultoria, e um relatório de engenharia, qualidade e testes, para engenheiros e gestores. O processo que produziu os dois (cerca de 60 agentes de análise e 17 de redação, volumes de mais de 250 e 160 páginas) foi longo demais para ser repetido de memória. Ele virou uma **skill**.

14.1 O que a skill produz

- **Volume I: proteção de dados.** Inventário de dados e operadores, direitos dos titulares, segurança, achados *verificados* e *refutados*, quesitos para o jurídico, plano de adequação. Não dá parecer: descreve fatos com evidência conferível e aponta dispositivos "possivelmente relacionados". A força é a precisão; um único fato errado desacredita o documento inteiro, e todo o método existe para impedir isso.
- **Volume II: engenharia, qualidade e testes.** Arquitetura, regras de codificação, catálogo de testes com nomes e resultados, avaliações de usabilidade com método GenderMag e persona de domínio, uso de IA no desenvolvimento, guia prático. Cada capítulo diz o quê, por quê, como foi aplicado, a evidência e a lição.
- Dois PDFs em LaTeX, dois pacotes para Overleaf e os papéis de trabalho (dossiês, refutações, registro de achados em JSON) que sustentam qualquer afirmação se alguém pedir a prova.

14.2 Como a skill está organizada

Arquivo	Quando o agente lê
`SKILL.md`	Sempre: mapa, passo zero com o usuário, regras para todo agente, fases, pedidos parciais, adaptações.
`references/metodo-lgpd.md`	Trilha de proteção de dados: contexto, gravidade, dimensões, prompts, consolidação, consultas agregadas.
`references/base-legal.md`	Base jurídica verificada, com retrato do regulador na data e armadilhas de citação.
`references/metodo-ux.md`, `engenharia.md`, `redacao.md`, `latex.md`	Catálogo de telas e avaliações; inventário do Volume II; roteiro de cada capítulo e prompts de redator; identidade visual e compilação.
`references/licoes.md`	Antes de orquestrar: o que deu errado na execução de referência (custo e interrupção, verdade, redação, ferramentas, relação com o usuário).
`assets/workflows/`, `assets/latex/`	Modelos de workflow multiagente parametrizados; estilo LaTeX, modelos dos dois volumes e diagramas.
`scripts/`	Preparar volumes, gerar registro de achados, gerar anexo de testes, renderizar páginas, detectar linhas estouradas, empacotar, limpar texto de lei, extrair PDF.

14.3 Três escolhas de método

- **Passo zero obrigatório.** Antes de gastar qualquer agente, a skill pergunta o que não dá para deduzir do repositório: quais volumes, quem pede e em que papel (dono dos dados, desenvolvedor contratado, fornecedor multi-cliente), identificação de capa, quem são os titulares, ambientes e acesso. Isso muda papéis jurídicos, abertura e capítulos inteiros.
- **Verificação adversarial.** Cada achado de proteção de dados passa por um agente que tenta refutá-lo com evidência do código ou do banco. Só o que sobrevive entra como "verificado"; o que cai entra como "refutado", com a prova. Um documento jurídico não pode afirmar conformidade sem prova nem apontar problema que não existe.
- **Separar análise de redação.** Os agentes de análise produzem dossiês estruturados; os de redação escrevem capítulos a partir dos dossiês, com passes de edição. Misturar os dois produzia texto bonito sobre fatos não conferidos.

14.4 Como foi criada e testada

A skill foi extraída com a skill de criação de skills: capturar a intenção a partir do histórico da própria execução, escrever o rascunho, rodar casos de teste com e sem a skill, comparar. O teste decisivo foi um *dry-run* contra uma stack diferente (outra linguagem, outro banco) para tirar do texto tudo o que era específico do projeto de origem. O resultado é um pacote `.skill` instalável em qualquer máquina. Limitação honesta: ela foi testada a seco, sem uma execução completa em outro projeto.

15. Lições aprendidas

Cada item abaixo custou pelo menos uma sessão e por isso está no CLAUDE.md, em `docs/mobile-rules.md` ou na memória do agente.

- **Rota nova quebra o typecheck até reiniciar o servidor de desenvolvimento.** Rotas tipadas são geradas no start do bundler; não se "conserta" o erro mudando o código da rota.
- `EXPO_PUBLIC_*` **é inlinado em build time.** Mudar o `.env` não muda um app já buildado.
- **Diretórios nativos são gerados.** Editar lá evapora no próximo prebuild; a mudança vai no config ou num plugin.
- **Dois ambientes e um CLI linkado.** Confirmar para qual ambiente se está apontando antes de qualquer escrita; o plano de testes exige provar o alvo todo dia.
- **Arquivo de tipos gerado já foi commitado vazio.** Um redirecionamento de shell truncava o arquivo antes de o gerador falhar. O script atual só grava se a saída tiver o tipo esperado.
- **Componentes nativos em beta exigem dev client novo a cada versão.** O componente que os usa carrega por `require` condicional e cai num fallback em JavaScript.
- **Edge-to-edge no Android.** Padding fixo na base não é área segura; o ajuste automático de inset é só do iOS.
- **Tokens de fundo de página não servem para tiles.** No tema escuro, o tile afunda em vez de subir.

- **Codificação ao editar por shell no Windows.** Um cmdlet de escrita corrompeu acentos num arquivo de código-fonte; o teste de texto pegou. Edições de texto com acento passam por ferramenta de edição, não por pipeline de shell.
- **Versão fechada na loja.** Um build de homologação com a mesma versão de marketing de um build já aprovado é recusado; a versão precisa voltar da branch principal para a de desenvolvimento após cada release.
- **Função SQL inlinada pula a checagem de permissão em plano reaproveitado.** plpgsql quando a função é portão de acesso.

16. O que levar deste caso

1. **Escreva o contrato antes do código.** Um CLAUDE.md com invariantes, fronteiras e imposição explícita é o que transforma um agente genérico em um colega que conhece o projeto.
2. **Imponha por máquina o que der.** Lint para tokens e imports, teste para texto legal, gate para dependências, policy para autorização. O que sobra como convenção, nomeie como convenção.
3. **Dê a cada agente um papel e uma negativa.** "Não use para X; isso é do agente Y" evita mais erros do que qualquer lista de capacidades.
4. **Trate o histórico do git como registro de decisões.** Sem comentários no código, o porquê mora na mensagem de commit, e ela nunca envelhece em relação ao código.
5. **Registre a armadilha no mesmo dia.** Cada item da seção 15 foi escrito na sessão em que custou tempo. É a parte do projeto que mais economiza.
6. **Faça os agentes falarem entre si quando o contrato cruza repositórios.** Ferramentas como o papo tiram o humano do papel de proxy e deixam para ele só a decisão.
7. **Transforme processos longos em skills.** O que um time fez uma vez com sessenta agentes vira um pacote que outro time roda com um pedido, desde que tenha sido testado fora do contexto de origem.

Este artigo descreve práticas de um projeto real, com identificadores, nomes e regras de negócio removidos. As ferramentas citadas são públicas: Expo, Supabase, Playwright, k6, semantic-release, GitHub Actions, Claude Code, Model Context Protocol e papo. Nenhuma afirmação aqui depende de dado confidencial para ser verificada.